

Dalsilens: A Real-Time Mobile Assistive Tool for Dichromacy Using LMS Daltonization and HSV-Based Color Identification

Anaf Naufalian
Indonesia
anafnaufalian@dalsicore.com

Abstract—Color Vision Deficiency (CVD), commonly known as color blindness, is a genetic condition that affects a person’s capacity ability to differentiate between certain light wavelengths. Although eyesight itself is usually normal, individuals with CVD find it difficult to distinguish colors within the red, green, and blue spectrums. The most common type is Dichromacy, which includes Protanopia (red-blind), Deutanopia (green-blind), and Tritanopia (blue-blind). Existing assistive technologies, such as color-filtering glasses, are designed to separate overlapping red and green wavelengths, help with color distinction but are often expensive. Additionally, many existing smartphone applications require users to upload images for processing, which prevents real-time operation and limits immediate visual feedback. This paper proposes the implementation of the LMS Daltonization algorithm on the Android platform using real-time image processing. The method converts the camera’s input from the RGB color space to the LMS color space. This allows for the mathematical simulation and adjustment of the missing color spectrums. The application is expected to help users distinguish objects with specific color spectrum through a smartphone, with low latency that supports real-time usage.

Index Terms—Color Vision Deficiency, Dichromacy, Daltonization, LMS Color Space, Real-Time Image Processing, Android.

I. INTRODUCTION

Vision is the primary sense in humans that plays a role in processing information from the external environment. Several studies mention that 80% of the information received and processed by the human brain comes from the visual system. This indicates that the sense of sight plays a crucial role in understanding the surrounding environment [1]. However, not everyone has this ability due to Color Vision Deficiency (CVD) or color blindness. Individuals with color blindness (particularly Protanopia and Deutanopia) often have difficulty distinguishing certain colors such as red (Protanopia) and green (Deutanopia) [2]. This difficulty can affect an individual’s ability to differentiate colors such as traffic lights, color codes on electronic devices, and even liquids. Various solutions to this problem are already available, one of which is chromatic glasses specifically designed to filter certain overlapping wavelengths of light between red and green [3]. However, the use of chromatic glasses still has some limitations, such as their relatively high price and limited availability.

As mobile technology advances, smartphones now have high computational capabilities to perform digital image pro-

cessing (Digital Image Processing) in real-time. Based on StatCounter data, one of the most widely used platforms worldwide is Android. The Android operating system dominates the global mobile device market with a percentage of 71.68% in 2025 [4]. The Android platform also provides a built-in library for manipulating color matrices, known as the Color Matrix, which allows for efficient pixel modification on images without affecting device performance.

Based on the previous problem, this research aims to design and implement a visual aid application titled “Dalsilens: A Real-Time Mobile Assistive Tool for Dichromacy Using LMS Daltonization and HSV-Based Color Identification”. This application is designed to help colorblind individuals distinguish colors on an object by using the smartphone camera in real-time. One of the algorithmic approaches applied in this research is the LMS Daltonization method. This method works by converting the RGB color matrix into the LMS color space, which represents the response of cone cells in the human eye. Next, the system calculates the missing color information in individuals with dichromacy and shifts that information to another color spectrum that can still be seen, thereby enhancing visual contrast without altering the structure or shape of the original object. This solution is expected to serve as an alternative to software-based assistive devices that are on-device, practical, and more accessible compared to conventional optical solutions.

II. RELATED WORK

In recent years, various approaches and technological solutions have been developed to assist individuals with Color Vision Deficiency (CVD) in improving their color discrimination ability. For example, the authors in reference [5] conducted a smartphone-based comparison of four different algorithms (LMS Daltonization, Color Blind Filter Service (CBFS), LAB color corrector, and shifting color). Each algorithm was tested using a set of test images for each type of *Dichromacy*. The evaluation process was conducted using color blindness simulator software, namely Vischeck and Coblis, to compare the differences in color perception before and after the correction process, as well as to analyze the processing time required by each algorithm. However, the testing in that research is still based on static image processing uploaded manually. The developed application does not process images in real-time.

The researcher explained that users must point the camera, capture the image, and then press the "OK" button. After that, the application will process the image and display it on the smartphone screen.

Additionally, in the article [6], the researchers developed a mobile application that serves as an aid for enhancing color vision for individuals with Color Vision Deficiency (CVD). The application also utilizes the LMS Daltonization algorithm to transform image colors in real-time using a camera and implements the Farnsworth D-15 Color Arrangement Test to diagnose the type of color blindness the user has directly within the application. Although it successfully displays simulations in real-time, the matrix processing still relies on manual color value calculations. This sequential computational approach can burden CPU performance, especially on devices with high camera resolution.

Therefore, this research focuses on optimizing the efficiency of real-time visual computation by shifting the operational load from the CPU to the GPU to avoid heavy sequential computation. By utilizing the CameraX library and *Color-Matrix* manipulation, DaltonLens can achieve stable real-time color correction, thereby addressing the frame-drop and latency issues that often occur in conventional real-time image processing.

III. METHODOLOGY

A. System Architecture

Overall, the architecture of the Dalsilens system is designed to process images in real time with low latency. Where the computation is divided into two main stages, image acquisition for color detection and bitmap-level color manipulation using a color matrix and GPU-accelerated filtering.

1) *Image Acquisition and Color Identification*: The flowchart in Figure 1 illustrates the initial image processing of the images captured by the camera hardware. First, when the user accesses the main interface of the application, the system automatically initializes the camera module. Next, the camera continuously captures each frame and forwards it to the *ImageAnalysis* component. The image received by this component is then converted into a raw bitmap representation for further processing by the system memory. After the conversion process is complete, the system determines a specific area or Region of Interest (ROI) located right in the center of the bitmap, representing the object targeted by the crosshair. The system extracts a set of pixels within the ROI for analysis. The extracted pixels are averaged to obtain a representative RGB value, which is subsequently converted into the HSV color space. The resulting HSV values are then classified using a rule-based thresholding approach to determine the dominant color in the target area. At the final stage, the raw bitmap and the generated dominant color label are stored in a state variable. Changes in this state subsequently trigger reactive UI updates, enabling dynamic rendering of the visual output.

2) *Daltonization Process*: Based on the flowchart in Figure 2, after the bitmap and color labels are obtained and stored, the system triggers a reactive update mechanism to refresh the

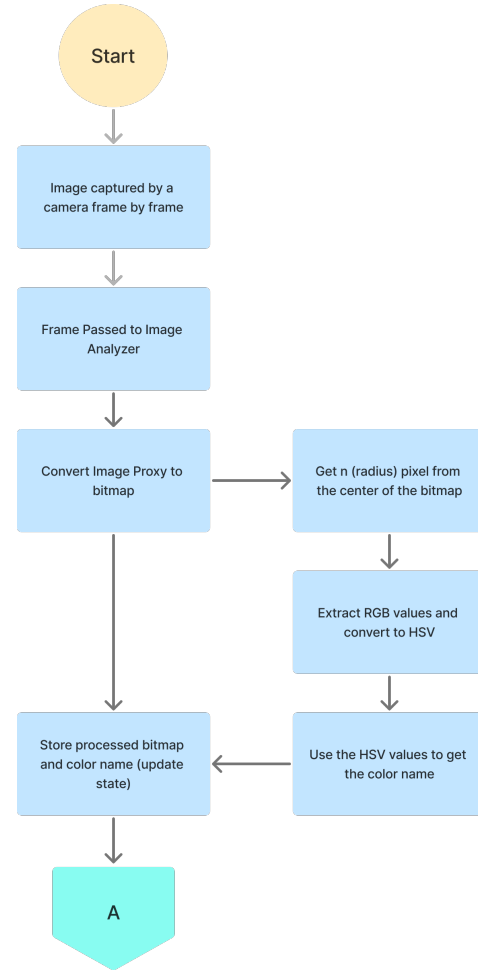


Fig. 1. Flowchart for image acquisition and color identification

user interface (UI). Changes in the detected color labels are immediately reflected in the UI, while the bitmap undergoes a verification stage to determine whether the filter is activated. If the filter is not activated, the system applies an identity color matrix to preserve the original color representation of the image. Conversely, if the filter is activated, a color matrix transformation is applied according to the selected color blindness simulation model. Furthermore, when the daltonization feature is enabled, the image is further processed using the LMS-based daltonization algorithm to enhance color perception. Next, the bitmap and the predetermined color matrix will be passed to the Image component. At this stage, color manipulation is performed using the Color Matrix Filter, which is executed through GPU acceleration, allowing the rendering process to be carried out efficiently.

B. Implementation of the LMS Daltonization Algorithm

In this section, we will discuss how to implement the LMS Daltonization algorithm, starting from converting RGB to LMS, which is the color space used by the human eye to perceive colors. Then we will simulate color blindness, up to the process of implementing the daltonization algorithm.

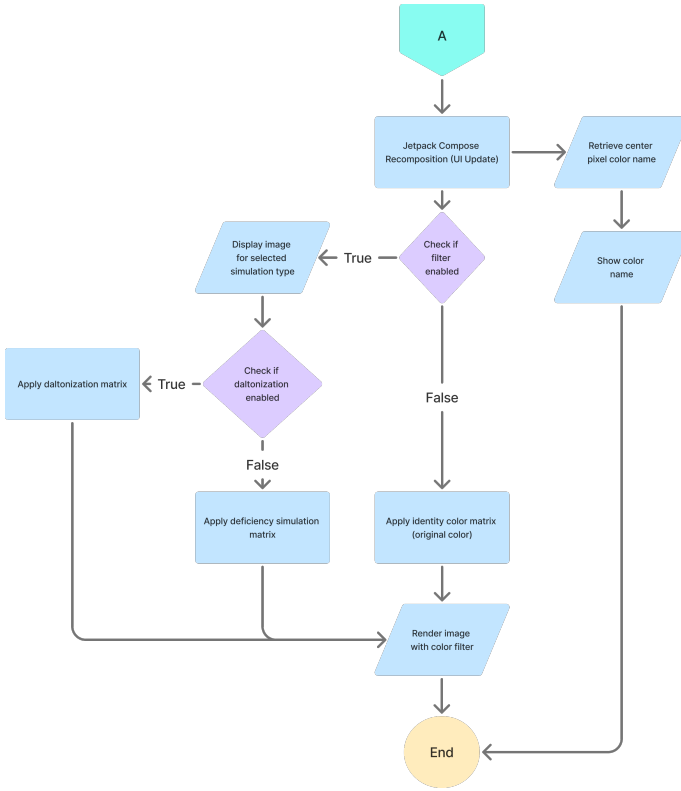


Fig. 2. Flowchart of Color Vision Deficiency Simulation and Daltonization Process

1) *RGB Color Space to LMS Conversion*: The first stage in the implementation process of the LMS Daltonization algorithm is to convert the color space of the image from RGB (Red, Green, Blue) to LMS (Long, Medium, Short). The conversion of the color space from RGB to LMS is performed using a matrix-based linear transformation, which represents the three types of cones in the human visual system, Long (L), Medium (M), and Short (S). Mathematically, this transformation is carried out by multiplying the RGB color vector with a specific conversion matrix, resulting in a color representation in the LMS space. Each element in the LMS space is calculated as a linear combination of the R, G, and B components, reflecting the contribution of each color channel to the cone cell response. The calculation of the values L, M, and S in this study is expressed as follows:

$$\begin{bmatrix} L \\ M \\ S \end{bmatrix} = \begin{bmatrix} 17.8824 & 43.5161 & 4.11935 \\ 3.4565 & 27.1554 & 3.86714 \\ 0.02996 & 0.18431 & 1.46709 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} \quad (1)$$

The coefficients in the equation were obtained from [7], which serves as a guideline in the color conversion process from the RGB space to LMS. These values indicate the relationship between RGB elements and the response of each channel in the LMS space so that the conversion results can be accurately applied in the later processing stage. The result of this conversion process consists of three main components that is L, M, and S. The use of the LMS color space at this

stage allows color representation to be processed in a form that aligns with the human vision model. This color space is used as the basis for simulating color vision deficiencies and applying the daltonization algorithm in the later stages.

2) *Simulation of Color Vision Deficiency*: The second stage focuses on mathematical modeling to simulate the decline in color perception ability based on the type of color vision deficiency, namely protanopia, deuteranopia, and tritanopia. This process is carried out by modifying the component values in the LMS space obtained in the previous stage. The simulation is conducted by adjusting one of the components in the LMS space according to the characteristics of each type of deficiency. In this condition, one of the components (L, M, or S) loses its sensitivity and is not used directly but recalculated based on the combination of the other two components. Mathematically, the transformation for each type of deficiency is expressed as follows:

Protanopia

$$\begin{bmatrix} L_{sim} \\ M_{sim} \\ S_{sim} \end{bmatrix} = \begin{bmatrix} 0 & 2.02344 & -2.52581 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} L \\ M \\ S \end{bmatrix}$$

Deuteranopia

$$\begin{bmatrix} L_{sim} \\ M_{sim} \\ S_{sim} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0.494207 & 0 & 1.24827 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} L \\ M \\ S \end{bmatrix} \quad (2)$$

Tritanopia

$$\begin{bmatrix} L_{sim} \\ M_{sim} \\ S_{sim} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -0.395913 & 0.801109 & 0 \end{bmatrix} \begin{bmatrix} L \\ M \\ S \end{bmatrix}$$

The coefficients in each equation were obtained from [7], which are used to adjust the changes in color components according to the type of deficiency being simulated. In this way, the simulation results can show how colors appear to users with specific color vision deficiencies. The results of this stage are modified LMS values to simulate vision under color deficiency conditions. These values are used to represent how colors are seen by individuals with color blindness, as well as to observe changes in results after applying the deuteranopia algorithm.

3) *Daltonization Process*: At this stage, the process of daltonization is carried out by calculating the difference between the original color and the simulated color deficiency result. The calculation begins by converting the color values from the RGB color space to LMS, then simulating the color deficiency in the LMS space, and subsequently converting it back to the RGB color space. In the previous stage, the simulated LMS values were obtained. To calculate the error value, the simulation result needs to be converted back to the RGB color space. The conversion process from LMS to RGB follows the equation below, with coefficients obtained based on [7].

$$\begin{bmatrix} R_{sim} \\ G_{sim} \\ B_{sim} \end{bmatrix} = \begin{bmatrix} 0.080944 & -0.130504 & 0.116721 \\ -0.0102485 & 0.0540194 & -0.113615 \\ -0.000365294 & -0.00412163 & 0.693513 \end{bmatrix} \begin{bmatrix} L_{sim} \\ M_{sim} \\ S_{sim} \end{bmatrix} \quad (3)$$

Next, the error value is calculated by determining the difference between the original RGB value and the simulated RGB value.

$$\begin{aligned} E_R &= R - R_{sim} \\ E_G &= G - G_{sim} \\ E_B &= B - B_{sim} \end{aligned} \quad (4)$$

The obtained error value is then redistributed to other color components through the *error shifting* process. This process is carried out because the color components affected by the deficiency cannot be perceived optimally by the user. Therefore, the missing color information is redirected to other color components that can still be distinguished, so that the visual information can be preserved. The error redistribution process is carried out using different transformation matrices for each type of color deficiency. This difference aims to adjust the direction of error distribution to the color components that can still be optimally perceived by the user.

In the condition of protanopia, the red component experiences a decrease in sensitivity, resulting in it not receiving the error distribution. Therefore, all error information is redirected to the green and blue components. The coefficients in this transformation matrix were obtained from [8], which are used to adjust the proportion of error redistribution to maintain visual differences between colors.

Protanopia

$$\begin{bmatrix} R_{shift} \\ G_{shift} \\ B_{shift} \end{bmatrix} = \begin{bmatrix} 0.0 & 0.0 & 0.0 \\ 0.9 & 0.1 & 0.0 \\ 0.9 & 0.0 & 0.1 \end{bmatrix} \begin{bmatrix} E_R \\ E_G \\ E_B \end{bmatrix} \quad (5)$$

In the condition of deuteranopia, the green component is the most affected component and is therefore not used in the error redistribution process. Instead, the error is distributed to the red and blue components. The coefficients in this matrix are obtained from [9].

Deuteranopia

$$\begin{bmatrix} R_{shift} \\ G_{shift} \\ B_{shift} \end{bmatrix} = \begin{bmatrix} 1.0 & 0.7 & 0.0 \\ 0.0 & 0.0 & 0.0 \\ 0.0 & 0.7 & 1.0 \end{bmatrix} \begin{bmatrix} E_R \\ E_G \\ E_B \end{bmatrix} \quad (6)$$

In the condition of tritanopia, the blue component experiences a decrease in sensitivity, resulting in it not receiving the error distribution. Therefore, the error is redirected to the red and green components. The coefficient values in the transformation matrix were obtained from [10].

Tritanopia

$$\begin{bmatrix} R_{shift} \\ G_{shift} \\ B_{shift} \end{bmatrix} = \begin{bmatrix} 1.0 & 0.0 & 0.7 \\ 0.0 & 1.0 & 0.7 \\ 0.0 & 0.0 & 0.0 \end{bmatrix} \begin{bmatrix} E_R \\ E_G \\ E_B \end{bmatrix} \quad (7)$$

After obtaining the error shift value, the next step is to add that value to the original color. This process results in a new adjusted color, making previously hard to see color information more distinguishable.

$$\begin{aligned} R_{dalton} &= R + R_{shift} \\ G_{dalton} &= G + G_{shift} \\ B_{dalton} &= B + B_{shift} \end{aligned} \quad (8)$$

4) *Construction of a 4x5 Transformation Matrix:* At this stage, the entire color transformation process described in Equation 8 is represented as a transformation function $T(R, G, B)$. To optimize the computational load and avoid repeated pixel-level calculations, this transformation function is simplified into a constant matrix of size 4×5 . This approach is carried out by evaluating the function T against three primary color vectors: pure red (1, 0, 0), pure green (0, 1, 0), and pure blue (0, 0, 1).

$$\begin{aligned} T(1, 0, 0) &= (r_R, r_G, r_B) \\ T(0, 1, 0) &= (g_R, g_G, g_B) \\ T(0, 0, 1) &= (b_R, b_G, b_B) \end{aligned} \quad (9)$$

The values resulting from the transformation of the basis vectors are then distributed into the coefficient columns of the matrix M_{dalton} . Through this arrangement, the matrix can map the original color profile to a new color that has been directly compensated, while also maintaining the *alpha* (transparency) parameter in the fourth row.

$$M_{dalton} = \begin{bmatrix} r_R & g_R & b_R & 0 & 0 \\ r_G & g_G & b_G & 0 & 0 \\ r_B & g_B & b_B & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \quad (10)$$

C. Implementation of Severity Level

This section explains how to implement the dynamic adjustment of color deficiency severity. The severity parameter is represented in the decimal range of 0.0 to 1.0. To maintain the accuracy of visual representation, the system differentiates the method of applying severity based on its purpose, namely between the vision simulation process and the daltonization process.

1) *Severity Modeling in Deficiency Simulation:* In simulation mode, the severity parameter serves to mimic the level of damage or shift in sensitivity in the user's cone cells. This process approximates a medical condition known as Anomalous Trichromacy [11]. Therefore, the calculations are performed directly in the LMS color space using linear interpolation methods. The system calculates the difference between the LMS values of normal vision and the LMS values under full deficiency (total) conditions. This difference is then multiplied by the *severity* parameter and added back to the normal value. The mathematical equation for interpolation in the LMS color space is stated as follows:

$$\begin{aligned}
L_{final} &= L_{normal} + (L_{sim} - L_{normal}) \times S \\
M_{final} &= M_{normal} + (M_{sim} - M_{normal}) \times S \\
S_{final} &= S_{normal} + (S_{sim} - S_{normal}) \times S
\end{aligned} \quad (11)$$

2) *Severity Modeling in Color Vision Correction*: Different from the simulation stage, the severity parameter in the daltonization process serves to determine the intensity or strength of the color assistance provided to the user. This calculation is executed in the RGB color space at the final stage of computation, specifically when adding the error shift value to the original pixel. The intensity of color compensation is adjusted by multiplying the error shift results from each color channel by the severity value before merging with the original color. This approach ensures that users can adjust color contrast proportionally according to their visual comfort level. The intensity adjustment equation is formulated as follows:

$$\begin{aligned}
R_{dalton} &= R_{asli} + (Shift_R \times S) \\
G_{dalton} &= G_{asli} + (Shift_G \times S) \\
B_{dalton} &= B_{asli} + (Shift_B \times S)
\end{aligned} \quad (12)$$

D. Dominant Color Extraction Based on the HSV Color Space

In this subsection, we discuss the process of extracting the dominant color based on the HSV color space. As shown in the flowchart in Figure 3, this stage aims to extract the color from the object located at the midpoint targeted by the *crosshair* in *real-time*. This process begins with obtaining an image in bitmap form, followed by processing the pixel area around the midpoint, and finally classifying the color name using the HSV color space.

1) *Bitmap Acquisition and Processing*: Frames from the camera are obtained through *ImageAnalysis* on CameraX, which continuously captures images. These frames are then passed to the *analyzer* function, which produces an *ImageProxy*. Next, the *ImageProxy* is converted into a *Bitmap*. Considering that the camera sensor orientation varies across different Android devices, the *rotationDegrees* value from the frame's metadata is checked. If the rotation is not equal to zero, the bitmap will be rotated using a transformation matrix.

2) *Pixel Sampling*: At this stage, the color sampling process is not only done at a single pixel point in the center of the screen. Sampling just one pixel tends to be less effective because the color in an area can be influenced by noise, lighting, or other conditions. Therefore, a sampling approach is used by taking several pixels around the central point. By considering more than one pixel, the obtained color becomes more stable and can better represent the dominant color of the observed area.

The first step is to determine the radius of the sampling area, with the default radius value being $r = 5$. With that radius, the sampled area includes 121 pixels (11 x 11). The number of pixels used can be calculated using the following equation:

$$N = (2r + 1) \times (2r + 1) \quad (13)$$

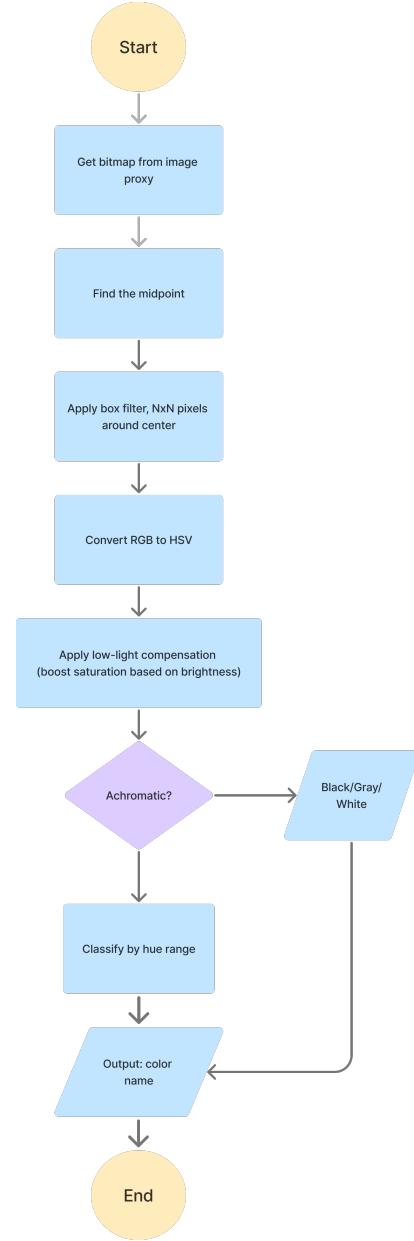


Fig. 3. Flowchart of Real-Time Color Extraction and Classification

Next, suppose there is an image I with width W and height H , and a center point (x_c, y_c) with radius R . To ensure that the processed pixel coordinates remain within the image boundaries, a bounding function is defined. This function is used to limit the coordinate values (x, y) so that they do not exceed the range $[0, W-1]$ and $[0, H-1]$.

$$\begin{aligned}
f(x) &= \min(\max(x, 0), W - 1) \\
g(y) &= \min(\max(y, 0), H - 1)
\end{aligned} \quad (14)$$

Each pixel at the coordinates (x, y) is stored as a 32-bit integer in *ARGB* format. The bit structure of the blue component (B) is in the last position (rightmost bit). Therefore, there is no need for a bit shift process; instead, it is sufficient

to perform masking to eliminate the other bits on its left using a mask value of 255 (0xFF).

$$B_{(x,y)} = \text{pixel} \& 255 \quad (15)$$

Meanwhile, the green color component (G) is in the middle position. Therefore, in order to be processed, the value needs to be shifted to the right by eight bits (shift right 8) to be in the last position, and then masking is performed to remove the other bits.

$$G_{(x,y)} = (\text{pixel} \gg 8) \& 255 \quad (16)$$

Finally, the red color component (R) is in the far left position. Therefore, the value needs to be shifted right by 16 bits to be in the last position, and then masking is performed to remove the other bits.

$$R_{(x,y)} = (\text{pixel} \gg 16) \& 255 \quad (17)$$

After the total RGB value is obtained, the next step is to calculate the average RGB value. The total RGB value is divided by the number of pixels in the sampling area (N). The average value calculation is performed using the following equation:

$$\begin{aligned} \bar{R} &= \frac{1}{N} \sum_{i=1}^N R_i \\ \bar{G} &= \frac{1}{N} \sum_{i=1}^N G_i \\ \bar{B} &= \frac{1}{N} \sum_{i=1}^N B_i \end{aligned} \quad (18)$$

3) *Color Name Classification*: The final stage is color classification, which is done by converting the obtained RGB values into the HSV color space. This conversion uses the built-in Android function in the *android.graphics.Color* class, namely *RGBToHSV(r, g, b, hsv)*. The conversion results in three components that are *Hue* (H) as the type of color in the range of 0°–360°, *Saturation* (S) as the level of color intensity in the range of 0–1, and *Value* (V) as the level of brightness in the range of 0–1. These three components are then used to determine the color name using a *threshold* approach based on the ranges of H, S, and V values. The *threshold* values are determined through testing color samples under normal and low-lighting conditions, considering the mean, standard deviation, and tolerance range.

For example, here is the calculation for determining the H value range for the color red. The test data were obtained from 10 samples under normal lighting conditions and 10 samples under low-lighting conditions, with the following results.

The data in the table is then used to calculate the mean value and the range of variation of the Hue component. The first step is to calculate the mean using the following formula:

TABLE I
HSV VALUES OF RED COLOR UNDER DIFFERENT LIGHTING CONDITIONS

Lighting Condition	H	S	V
Normal	3	126	190
Normal	3	126	191
Normal	4	128	191
Normal	3	126	191
Normal	3	126	191
Normal	4	126	191
Normal	3	126	191
Normal	3	126	191
Normal	3	127	190
Normal	4	127	194
Low	8	153	148
Low	8	153	146
Low	10	154	147
Low	8	153	146
Low	8	154	147
Low	9	155	148
Low	7	149	147
Low	8	119	143
Low	7	151	145
Low	8	157	139

$$\bar{H}_{\text{normal}} = \frac{1}{N_{\text{normal}}} \sum_{i=1}^{N_{\text{normal}}} H_i \quad (19)$$

$$\bar{H}_{\text{low}} = \frac{1}{N_{\text{low}}} \sum_{i=1}^{N_{\text{low}}} H_i \quad (20)$$

After the average value is obtained, calculations are performed for each lighting condition, normal and low. The calculation results show that the average value of the *Hue* component under normal lighting conditions is 3.3°, while under low lighting conditions it is 8.1°. Next, the standard deviation is calculated to determine the data distribution for each condition. This calculation aims to see the extent of the variation in *Hue* values relative to their average, so it can be used as a basis for determining tolerance limits. The next step is to calculate the standard deviation using the following formula:

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2} \quad (21)$$

After the standard deviation value is obtained, the tolerance value (upper and lower limits) can be determined by adding and subtracting the mean value with the standard deviation. Based on the calculation results, a standard deviation value of 0.48° was obtained under normal lighting conditions and 0.88° under low-lighting conditions. This value will be used to determine the upper and lower limit ranges of the *Hue* component. To find the upper and lower limits, the following formula can be used:

$$\text{Tolerance Range} = \bar{H} \pm 2\sigma \quad (22)$$

Based on the equation, the tolerance limits for normal lighting conditions can be determined. The lower bound value

is obtained by subtracting two times the standard deviation from the average, which is $3.3 - 2(0.48^\circ) = 2.34^\circ$. Meanwhile, the upper bound is obtained by adding two times the standard deviation to the average, which is $3.3 + 2(0.48^\circ) = 4.26^\circ$. Therefore, the tolerance range for the *Hue* component under normal lighting conditions is between 2.34° and 4.26° .

Next, the same method is applied to low-lighting conditions. The lower-bound value is calculated using the equation $8.1 - 2(0.88^\circ) = 6.34^\circ$, and the upper-bound value is calculated using $8.1 + 2(0.88^\circ) = 9.86^\circ$. Therefore, the tolerance range of the *Hue* component under low-lighting conditions is within the interval 6.34° to 9.86° .

If compared, the range of *Hue* values from normal and low-lighting conditions does not show any overlap. Based on these results, the determination of the *threshold* value is done by setting the upper limit of the worst condition or the one with greater variation, which is 9.86° . Therefore, the *threshold* value for the color red can be expressed as:

$$H < 9.86^\circ$$

The same method is also applied to other colors using the test data for each. For each color, the average value, standard deviation, and determination of tolerance limits are calculated based on normal and low-lighting conditions. The final result of this process is summarized in the form of a threshold value table for each color, so it can be directly used in the color classification process.

TABLE II
COLOR CLASSIFICATION BASED ON HUE RANGE

Hue Range ($^\circ$)	Color Category
$H < 9.85$	Red
$9.85 \leq H < 26.98$	Orange
$26.98 \leq H < 50.42$	Yellow
$50.42 \leq H < 122.50$	Green
$122.50 \leq H < 204.85$	Cyan
$204.85 \leq H < 224.57$	Blue
$224.57 \leq H < 319.37$	Purple
$H \geq 319.37$	Red

It should be noted that not all colors can be classified using *Hue* because they do not have sufficient brightness levels. In certain conditions, especially when the *Saturation* value is low, colors tend to lack clear color characteristics and are closer to neutral colors (grayscale). Therefore, a check is performed with the *Saturation* value. If the *Saturation* value is low, the color classification no longer uses *Hue*, but is determined based on the *Value* according to Table III.

TABLE III
GRAYSCALE CLASSIFICATION BASED ON SATURATION AND VALUE

Condition	Value Range	Color Category
$S < 0.10$	$V < 0.15$	Black
$S < 0.10$	$0.15 \leq V < 0.40$	Dark Gray
$S < 0.10$	$0.40 \leq V < 0.78$	Gray
$S < 0.10$	$V \geq 0.78$	White

By combining color classification using the values of *Hue*, *Saturation*, and *Value*, the system can recognize colors better.

Once the *threshold* has been determined, the system can recognize and display color names in *real-time* even under varying lighting conditions. To evaluate the accuracy level of the color classification results, testing will be conducted in the next stage.

E. Testing Scenario

This section discusses how testing is conducted to evaluate the functionality, accuracy, and performance of the Dalsilens application. This evaluation is conducted to ensure that the implemented algorithm can provide real-time visual assistance for individuals with dichromacy in accordance with the research goals. In general, the system testing parameters in this research are divided into three main scenarios that are visual correction testing, color detection accuracy, and computational performance.

1) *Visual Correction Testing with Ishihara Plates*: The first scenario aims to measure the effectiveness of the LMS Daltonization algorithm in restoring lost color contrast. This test uses the medical color blindness test standard, namely the Ishihara plates. The system will process the images of the Ishihara plates into three conditions that are normal vision, simulated dichromacy deficiency (Protanopia, Deuteranopia, and Tritanopia), and the final result after daltonization correction is applied. Evaluation is conducted by observing whether numbers or patterns that were previously challenging to see become clearer after the colors are corrected.

2) *Testing the Accuracy of Color Name Classification*: The second scenario is designed to evaluate the accuracy level of the textual color identification feature based on the HSV color space. The testing is conducted by directing the device's camera at various object samples in real-world environments that represent different color spectrum ranges. To ensure the reliability of the system, color sampling was conducted under two different lighting conditions, namely normal lighting (bright) and low lighting (dim). The results of the color name classification from the application are then compared with the actual color of the object. The success rate of the system is measured in terms of accuracy percentage based on the number of correct detections compared to the total trials.

3) *Real-Time Performance Testing*: This scenario aims to validate the efficiency of visual computation driven by GPU acceleration. Performance testing is measured quantitatively by calculating the average *Frame Per Second* (FPS) value and processing latency when the system simultaneously performs color matrix transformations and calculations. Additionally, monitoring was also conducted using the *Android Studio Profiler* instrument on physical devices to ensure the stability of memory (RAM) usage and CPU utilization, thereby avoiding issues such as *lag* or *memory leak*.

IV. RESULT AND DISCUSSION

todo: tulisan

REFERENCES

- [1] S. Chung and J.-W. Son, "Visual perception in autism spectrum disorder: A review of neuroimaging studies," *Soa Chongsoryon Chongsin Uihak*, pp. 105–120, 2020.
- [2] H.-Y. Lin, L.-Q. Chen, and M.-L. Wang, "Improving discrimination in color vision deficiency by image re-coloring," *Sensors*, vol. 19, no. 10, 2019. [Online]. Available: <https://www.mdpi.com/1424-8220/19/10/2250>
- [3] E. Patterson, R. Mastey, J. Kuchenbecker, J. Rowlan, J. Neitz, M. Neitz, and J. Carroll, "Effects of color-enhancing glasses on color vision in congenital red-green color deficiencies," *Optics Express*, pp. 31 182–31 194, 2022.
- [4] StatCounter, "Mobile operating system market share worldwide," <https://gs.statcounter.com/os-market-share/mobile/worldwide/2025>, 2025, accessed: 2026-02-26.
- [5] L. A. Elrefaei, "Smartphone based image color correction for color blindness," *International Journal of Interactive Mobile Technologies (IJIM)*, vol. 12, no. 3, pp. 104–119, Jul. 2018. [Online]. Available: <https://online-journals.org/index.php/i-jim/article/view/8160>
- [6] G. Tecson, F. Calanda, G. Cayabyab, and F. Reyes Jr, "Covisance: A real time mobile recolorization tool for aiding color vision deficient users utilizing d-15 color arrangement test," 12 2017, pp. 95–99.
- [7] F. Viénot, H. Brettel, and J. D. Mollon, "Digital video colourmaps for checking the legibility of displays by dichromats," *Color Research and Application*, vol. 24, pp. 243–252, 1999. [Online]. Available: <https://api.semanticscholar.org/CorpusID:62166956>
- [8] A. Tasnim and M. S. Hasan, "An improved dynamic daltonization for color-blinds," in *2017 IEEE Region 10 Humanitarian Technology Conference (R10-HTC)*, 2017, pp. 798–801.
- [9] A. B. Muhammad, J. Z. Maitama, A. Bature, and Z. Yunusa, "Analysis of LMS daltonization algorithm for adjusting image color for deuteranopia," *Zaria Journal of Electrical Engineering Technology*, vol. 9, no. 1, pp. 128–135, March 2020.
- [10] J. Schmitz, "Color blindness simulation research," <https://ixora.io/projects/colorblindness/color-blindness-simulation-research/>, 2026, accessed: 2026-03-26.
- [11] G. M. Machado, M. M. Oliveira, and L. A. F. Fernandes, "A physiologically-based model for simulation of color vision deficiency," *IEEE Transactions on Visualization and Computer Graphics*, vol. 15, no. 6, pp. 1291–1298, 2009.